

Unresolved reference 'protected'.
Property getter or setter expected
Property must be initialized or be abstract.
Property "dataSource" is never used
Property "pythonClient" is never used
Property "domainRuleRepository" is never used
Property "DBdata" is never used
Remove redundant qualifier name
Parameter "e" is never used
Parameter "e" is never used
Function "printRow" is never used
Class "Value" is never used
Download grammar and spelling checker for German?
An interpolation prefix can simplify the string
An interpolation prefix can simplify the string
Property name 'DBdata' should start with a lowercase letter
Java-style call can be replaced with Kotlin extension
Java-style call can be replaced with Kotlin extension

```
@Component
class StartupCheckRunner(private val dataSource: DataSource,
private val pythonClient: PythonAnalysisClient) : ApplicationRunner {
    private val restTemplate = RestTemplate()
    @Value("\${python.api.base-url:http://localhost:8000}")
    private val pythonBaseUrl: String = "\${python.api.base-
url:http://localhost:8000}"

    private var domainRuleRepository: DomainRuleRepository =

    protected var DBdata: DataSource // Für den Ping-Test
```

Nachgedacht für 7s >

Du hast hier eigentlich **zwei verschiedene Probleme**: (1) **echte Syntax-/Kompilierfehler** in deinem Kotlin-Snippet und (2) **IDE-Warnungen/Inspections** (unused, redundant qualifier, "interpolation prefix ...", etc.).

1) Warum „Unresolved reference 'protected'“ + „Property getter or setter expected“ passiert

In deinem Code steht:

```
kotlin

private var domainRuleRepository: DomainRuleRepository =

    protected var DBdata: DataSource
```

Das ist **syntaktisch kaputt**, weil nach `=` eine Initialisierung erwartet wird. Stattdessen kommt eine neue Property-Deklaration. Dadurch „verliert“ der Parser den Kontext und dann kommen Folgefehler wie:

- **Property getter or setter expected**
- **Property must be initialized or be abstract**
- **Unresolved reference 'protected'** (weil die Stelle nicht mehr als Property-Deklaration geparkt wird, sondern als Quatsch nach einem `=`)

 Code kopieren

✅ Fix: `domainRuleRepository` **entweder korrekt injizieren** (am besten über den Konstruktor) oder **lateinit** verwenden, aber nicht halbfertig mit `=` stehen lassen.

2) Dein @Value ist falsch initialisiert

Du hast:

```
kotlin

@Value("\${python.api.base-url:http://localhost:8000}")
private val pythonBaseUrl: String = "\${python.api.base-url:http://localhost:8000}"
```

Hier setzt du den Wert **literally** auf den String `\${python.api.base-url...}` . Dadurch kommt auch die IDE-Warnung „An interpolation prefix can simplify the string“ usw.

 Code kopieren

✔ So macht man das richtig: **keine manuelle Initialisierung**, Spring setzt das.

3) Saubere Lösung (empfohlen): Alles per Konstruktor injizieren

So ist es idiomatisch in Kotlin + Spring und beseitigt 90% deiner Meldungen:

```
kotlin

import org.springframework.beans.factory.annotation.Value
import org.springframework.boot.ApplicationRunner
import org.springframework.stereotype.Component
import javax.sql.DataSource

@Component
class StartupCheckRunner(
    private val dataSource: DataSource,
    private val pythonClient: PythonAnalysisClient,
    private val domainRuleRepository: DomainRuleRepository,
    @Value("\${python.api.base-url:http://localhost:8000}")
    private val pythonBaseUrl: String
) : ApplicationRunner {

    override fun run(args: org.springframework.boot.ApplicationArguments) {
        // Beispiel: Ping-Test / Startup-Checks
        dataSource.connection.use { conn ->
            // do something lightweight
        }

        // pythonClient benutzen, sonst bleibt es "never used"
        // domainRuleRepository benutzen, sonst bleibt es "never used"
        // pythonBaseUrl benutzen, sonst bleibt es "never used"
    }
}
```

Warum das deine Warnungen reduziert

- Keine uninitialisierten Properties → kein „must be initialized“
- Kein `protected var DBdata` nötig (du hast bereits `dataSource`)
- Keine `@Value` -Fehlinitialisierung
- Weniger „Property ... is never used“, sofern du sie im `run()` wirklich nutzt

4) Wenn du unbedingt eine Property statt Konstruktor willst (geht auch)

```
kotlin

@Value("\${python.api.base-url:http://localhost:8000}")
lateinit var pythonBaseUrl: String
```

Aber: In Kotlin ist **constructor injection** fast immer besser.

5) Zu den restlichen IDE-Warnungen (kurz, was du damit machst)

- **Property "... is never used" / Function "... is never used" / Class "... is never used"**
→ Entweder wirklich verwenden oder löschen. (Oder Sichtbarkeit reduzieren / `@Suppress("unused")` wenn Spring/Reflection es nutzt.)
- **Parameter "e" is never used**
→ In Lambdas: ersetze `e ->` durch `_ ->` oder entferne den Parameter.
- **Remove redundant qualifier name**
→ IntelliJ meint sowas wie `this.foo` wo `foo` reicht.
- **Java-style call can be replaced with Kotlin extension**
→ z.B. `collection.stream()` → Kotlin `collection.asSequence()` / `map/filter` usw.
- **Property name 'DBdata' should start with a lowercase letter**
→ Kotlin-Konvention: `dbData` (aber ehrlich: brauchst du vermutlich gar nicht)

6) „Download grammar and spelling checker for German?“ – was ist das?

Das ist sehr wahrscheinlich ein IntelliJ/Android Studio Hinweis (Grazie / LanguageTool / JetBrains Proofreading). Wenn du deutsche Rechtschreibprüfung willst:

- **JetBrains IDE:** Settings → Editor → Proofreading / Spelling → Deutsch aktivieren
- Oder Plugin **LanguageTool** installieren (funktioniert super für Deutsch)

Wenn du mir sagst, ob du IntelliJ IDEA / Android Studio nutzt (und welche Version ungefähr), sag ich dir die exakten Klickpfade.

Wenn du mir den Rest der Klasse (inkl. `DomainRuleRepository`-Typ und was du beim Startup prüfen willst) reinkopierst, kann ich dir den `run()` so umbauen, dass **keine "unused"** mehr übrig bleiben und die Checks sauber/logisch sind.



 **bug rule_industrie_api.rtf**
Datei

was da los

Nachgedacht für ein paar Sekunden >

Kurz gesagt: **es sind mehrere Baustellen gleichzeitig**, und deshalb wirkt's wie „alles brennt“. In deinem Log sieht man (1) Kotlin-Service ist nicht erreichbar → Fallback wird gezogen, und (2) du hast beim Rumändern in Python ein paar **harte Syntax-/NameErrors** gebaut, die Uvicorn beim Reload jedes Mal wieder um die Ohren fliegen.

1) Kotlin „offline“ / Connection refused (Port 8080)

Das hier ist der Kern:

- `Failed to establish a new connection: [Errno 61] Connection refused`
- URLs wie `/api/v1/rules/industry-mappings`, `/role-mappings`, `/esco-full`
- Basis-URL: `http://localhost:8080`

Bedeutet: auf deinem Rechner läuft **nichts** auf `localhost:8080` (oder es läuft, aber nicht erreichbar). Postgres läuft nur auf `5432` (das sieht man in `docker ps`).

✅ Checkliste:

- Läuft dein Kotlin/Spring Boot überhaupt?
 - `curl http://localhost:8080/actuator/health` (oder dein Health-Endpoint)
- Läuft es evtl. in Docker und ist nicht auf 8080 gemappt?
 - `docker ps` → Ports anschauen (bei dir sieht man nur Postgres)
- Oder falsche URL/Env-Var:
 - Du nutzt `KOTLIN_API_URL` / Default `http://localhost:8080` (siehe Log). [bug rule_industrie_api](#)

2) Uvicorn Reload-Schleife: du triggerst Neustarts ständig

Du startest mit `--reload` , dann:

- „StatReload detected changes... Reloading...“
- Server fährt runter, startet neu, crasht wieder

Das passiert, weil beim Reload Import/Startup-Code sofort Fehler wirft. [bug rule_industrie_api](#)

✅ Quick fix zum Debuggen:

- Starte einmal **ohne** Reload, damit du den Fehler sauber siehst:
 - `uvicorn main:app` (ohne `--reload`)

3) Konkrete Python-Fehler, die dich wirklich killen

a) `NameError: name 'os' is not defined`

In `app/core/constants.py` greifst du auf `os.environ.get(...)` zu, aber hast `import os` vergessen.

[bug rule_industrie_api](#)

✅ Fix:

```
py

import os
```

b) `NameError: KOTLIN_API_BASE_URL is not defined`

In `kotlin_rule_client.py` setzt du `self.base_url = KOTLIN_API_BASE_URL` , aber diese Konstante existiert nicht (oder wurde umbenannt, z.B. zu `DEFAULT_KOTLIN_URL`). [bug rule_industrie_api](#) [Code kopieren](#)

✅ Fix-Idee:

- Ein einziges Source-of-truth in `constants.py` (z.B. `DEFAULT_KOTLIN_URL`)
- Und im Client importieren:

```
py

from app.core.constants import DEFAULT_KOTLIN_URL
self.base_url = DEFAULT_KOTLIN_URL
```

c) `NameError: response is not defined`

In `get_esco_full()` wird `response.raise_for_status()` aufgerufen, aber `response` wurde in dem Scope nicht gesetzt (wahrscheinlich in einem try/except versehentlich überschrieben/verschoben). [bug rule_industrie_api](#) [Code kopieren](#)

✅ Fix:

- Sicherstellen, dass `response = requests.get(...)` **vor** `raise_for_status()` steht und nicht in einem Zweig verloren geht.

d) `IndentationError nach except`

Du hast im `kotlin_rule_client.py` ein `except:` und danach ist der Block nicht korrekt eingerückt, dadurch crasht Python schon beim Parsen. [bug rule_industrie_api](#)

✔ Fix:

- In der Datei die `try/except` -Blöcke sauber einrücken (und am besten `except Exception as e:` statt blanko).

Was ist „das Los“ als Ursache-Kette?

1. Kotlin nicht auf 8080 erreichbar → Python versucht Regeln/ESCO zu holen → fällt teils auf Fallback zurück. `bug rule_industrie_api`
2. Du editierst während `--reload` läuft → jeder Save startet neu. `bug rule_industrie_api`
3. Beim Neustart crasht der Import wegen `os` fehlt / Konstante fehlt / response fehlt / Indentation kaputt → daher kommt die Reload-Hölle. `bug rule_industrie_api`

Minimaler „mach es wieder grün“-Plan (in 5 Minuten)

1. **Uvicorn ohne reload** starten (besserer Fehler-Fokus).
2. `import os` in `constants.py` fixen.
3. In `kotlin_rule_client.py` :
 - Base-URL-Konstante sauber definieren/importieren
 - `response` -Variable korrekt setzen
 - `IndentationError` beheben (das ist nicht optional)
4. Danach entscheiden: Kotlin wirklich starten **oder** bewusst „Standalone Mode“ nutzen (dann sollte dein Client nicht ständig 8080 anpingen).

Wenn du mir **die aktuellen Inhalte** von

- `app/core/constants.py`
 - `app/infrastructure/clients/kotlin_rule_client.py` (komplett, oder mindestens die Stellen um die genannten Fehler)
- reinkopierst, kann ich dir die beiden Dateien direkt „clean“ hinbiegen (mit sauberem Fallback, ohne Crash beim Import).

Quellen



```
import requests
import os
import json
import pandas as pd
from typing import Set, List, Dict
from tqdm import tqdm

# ZENTRALISIERUNG: Wir nutzen die Konstante aus deiner
constants.py
try:
    from app.core.constants import DEFAULT_KOTLIN_URL
except ImportError:
    DEFAULT_KOTLIN_URL = os.environ.get("KOTLIN_API_URL",
"http://localhost:8080")

class KotlinRuleClient:
    """
    Adapter (Client) für die Abfrage von Domänenregeln
    (Blacklist, Mappings, ESCO) vom Kotlin DomainRuleService.
    Inkl. robustem File-System-Fallback.
    """
```

```

def __init__(self, base_url=None):
    # 1. FIX: Nimm die URL, die übergeben wird. Wenn keine da ist,
    nimm Environment/Default.
    self.base_url = base_url if base_url else DEFAULT_KOTLIN_URL
    self.fallback_path = "data/fallback_rules"
    self.esco_source = "data/esco"

    # Sicherstellen, dass der Fallback-Ordner existiert
    os.makedirs(self.fallback_path, exist_ok=True)
    print(f"🔗 KotlinRuleClient initialisiert. Basis-URL:
    {self.base_url}")

def get_esco_full(self):
    endpoint = f"{self.base_url}/api/v1/rules/esco-full"
    filename = "esco_fallback.json"

    try:
        print(f"🔄 Lade ESCO-Daten von: {endpoint}")
        response = requests.get(endpoint, timeout=15)
        response.raise_for_status()

        raw_data = response.json()

        # --- 🧑‍🔧 DEBUGGING START ---
        print(f" 🔍 DEBUG-INFO:")
        print(f" 📌 HTTP Status: {response.status_code}")
        print(f" 📌 Datentyp: {type(raw_data)}")

        if isinstance(raw_data, list):
            print(f" 📌 Anzahl Elemente: {len(raw_data)}")
            if len(raw_data) == 0:
                print(f" ⚠️ ACHTUNG: Die Liste von Kotlin ist LEER!")
            else:
                print(f" 📌 Probe (Erstes Element): {raw_data[0]}")
                # Zeige die Keys des ersten Elements, um Tippfehler zu
                finden
                if isinstance(raw_data[0], dict):
                    print(f" 📌 Verfügbare Keys:
                    {list(raw_data[0].keys())}")
                elif isinstance(raw_data, dict):
                    print(f" ⚠️ ACHTUNG: Habe Dictionary statt Liste erhalten.
                    Keys: {list(raw_data.keys())}")
                else:
                    print(f" ⚠️ ACHTUNG: Unerwartetes Format: {raw_data}")
            # --- 🧑‍🔧 DEBUGGING ENDE ---

        # 1. FIX: Liste VOR der Schleife initialisieren!
        normalized = []

        # 2. FIX: tqdm hier nutzen
        # Hinweis: Wenn du tqdm hier nutzt, brauchst du es im
        Repository eigentlich nicht mehr,
        # aber doppelt hält besser (oder du entfernst es dort).
        from tqdm import tqdm

        for item in tqdm(raw_data, desc="🚀 Lade Skills (Client)",
        unit=" sk", ncols=100):
            # Prüfen auf Label
            label = item.get("preferredLabel") or item.get("label") or
            item.get("esco_label")

```



```
        if label:
            # 3. FIX: .append() nutzen statt überschreiben (=)
            normalized.append({
                "label": label,
                "uri": item.get("esco_uri") or item.get("uri"),
                "is_digital": item.get("is_digital", False),
                "level": item.get("level", 2)
            })

        print(f"✅ {len(normalized)} Skills geladen & normalisiert.")

        # Cache speichern
        self._save_to_json(filename, normalized)
        return normalized

    except (requests.exceptions.RequestException,
            ConnectionError) as e:
        print(f"❌ API-Fehler bei ESCO ({e}). Versuche Fallback-Datei...")
        return self._load_or_create_fallback(filename,
            generation_method=self._generate_esco_from_csv)

    def fetch_blacklist(self) -> Set[str]:
        """Holt Blacklist. Fallback: JSON Datei (erstellt aus Defaults)."""
        endpoint = f"{self.base_url}/api/v1/rules/blacklist"
        filename = "blacklist_fallback.json"

        try:
            response = requests.get(endpoint, timeout=5)
            response.raise_for_status()
            data = response.json()
            # Cache update
            self._save_to_json(filename, data)
            return set(data)
        except Exception as e:
            print(f"❌ API-Fehler bei Blacklist: {e}")
            # Nutzt _get_static_fallback_blacklist_as_list zur Generierung
            # der Datei, falls sie fehlt
            data = self._load_or_create_fallback(filename,
            generation_method=self._get_static_fallback_blacklist_as_list)
            return set(data) if data else set()

    def fetch_role_mappings(self) -> Dict[str, str]:
        """Holt Rollen-Mappings."""
        endpoint = f"{self.base_url}/api/v1/rules/role-mappings"
        filename = "role_mappings_fallback.json"

        try:
            response = requests.get(endpoint, timeout=5)
            response.raise_for_status()
            data = response.json()
            self._save_to_json(filename, data)
            return data
        except Exception as e:
            print(f"❌ API-Fehler bei Role-Mappings: {e}")
            return self._load_or_create_fallback(filename,
            generation_method=self._get_static_fallback_role_mappings)

    def fetch_industry_mappings(self) -> Dict[str, str]:
        """Holt Branchen-Mappings."""
        endpoint = f"{self.base_url}/api/v1/rules/industry-mappings"
```

```
filename = "industry_mappings_fallback.json"

try:
    response = requests.get(endpoint, timeout=5)
    response.raise_for_status()
    data = response.json()
    self._save_to_json(filename, data)
    return data
except Exception as e:
    print(f"❌ API-Fehler bei Industry-Mappings: {e}")
    return self._load_or_create_fallback(filename,
generation_method=self._get_static_fallback_industry_mappings)

# --- HELPER & GENERATOREN ---

def _load_or_create_fallback(self, filename, generation_method):
    """
    Generische Methode: Prüft ob Datei existiert.
    Wenn nicht: Ruft generation_method() auf und speichert das
    Ergebnis.
    Dann: Lädt Datei.
    """
    path = os.path.join(self.fallback_path, filename)
    print(f"⚠️ in Load or Create Fallback-Datei {filename} mit {path}.")
    # 1. Wenn Datei nicht da, generieren wir sie EINMALIG
    if not os.path.exists(path):
        print(f"⚠️ Fallback-Datei {filename} fehlt. Erstelle sie neu...")
        data = generation_method()
        self._save_to_json(filename, data)

    # 2. Datei laden
    if os.path.exists(path):
        try:
            with open(path, "r", encoding="utf-8") as f:
                print(f" -> Lade lokalen Fallback: {filename}")
                #return json.load(f)
                # SCHRITT A: Daten erst in eine Variable laden
                data = json.load(f)

            # SCHRITT B: Test-Ausgabe (Nur die ersten paar, damit
            das Terminal nicht explodiert)
            print(f" 👁️ DEBUG-CHECK: {type(data)}")

            if isinstance(data, list):
                print(f" 🇩🇪 Anzahl Einträge: {len(data)}")
                if data:
                    print(f" liefere Probe (Erster Eintrag): {data[0]}")
            elif isinstance(data, dict):
                print(f" 🇩🇪 Anzahl Keys: {len(data)}")
                # Zeige die ersten 2 Einträge als Probe
                first_two = list(data.items())[:2]
                print(f" liefere Probe: {first_two}")

            # SCHRITT C: Daten zurückgeben
            return data
        except Exception as e:
            print(f"❌ Fehler beim Lesen von {path}: {e}")

    return {} # Notfall-Return

def _load_fallback_esco(self, cache_file: str) -> Dict:
    """Lädt Fallback-Daten robust."""
```



```
# 1. Versuche Cache-Datei (schnell)
if os.path.exists(cache_file):
    try:
        with open(cache_file, "r", encoding="utf-8") as f:
            data = json.load(f)
            print(f"📁 Fallback: Cache geladen ({len(data)} Skills).")
            return data
    except json.JSONDecodeError:
        print("❌ Cache-Datei korrupt.")

# 2. Versuche CSV-Generator (langsam, aber rettet den Start)
print("⚙️ Fallback: Generiere aus lokalen CSV-Dateien...")
return self._generate_esco_from_csv(cache_file)

def _save_to_json(self, filename, data):
    path = os.path.join(self.fallback_path, filename)
    try:
        with open(path, "w", encoding="utf-8") as f:
            json.dump(data, f, ensure_ascii=False, indent=2)
    except Exception as e:
        print(f"⚠️ Konnte Fallback {filename} nicht speichern: {e}")

# --- DATEN-QUELLEN FÜR GENERIERUNG ---

def _generate_esco_from_csv(self):
    """Liest CSVs und baut das JSON-Objekt."""
    fallback_list = []
    csv_file = os.path.join(self.esco_source, "skills.csv")
    if os.path.exists(csv_file):
        try:
            df = pd.read_csv(csv_file, usecols=['preferredLabel',
'conceptUri'])
            for _, row in tqdm(df.iterrows(),desc="🚀 Indiziere Skills",
unit=" sk", ncols=100):
                fallback_list.append({
                    "label": str(row['preferredLabel']),
                    "uri": row['conceptUri'],
                    "is_digital": False
                })
        except Exception: pass
    return fallback_list

def _get_static_fallback_blacklist_as_list(self):
    return list({
        "kenntnisse", "fähigkeiten", "kommunikation", "deutsch",
"englisch",
        "team", "management", "analyse", "projektleitung",
"erfahrung",
        "und", "oder", "bei", "mit", "job", "suche"
    })

def _get_static_fallback_industry_mappings(self) -> Dict[str, str]:
    return {
        'Informationstechnologie': 'IT|Software|Entwicklung|Cloud',
        'Finanzen': 'Bank|Versicherung|Finance',
        'Gesundheit': 'Pflege|Klinik|Medizin'
    }

def _get_static_fallback_role_mappings(self) -> Dict[str, str]:
    return {
        'Entwickler': 'Developer|Software Engineer|Programmierer',
        'Manager': 'Leiter|Head of|Lead'
```

```

    }

    def _get_static_fallback_blacklist(self) -> Set[str]:
        """
        Statischer Fallback: Falls die DB/API nicht erreichbar ist, nutzen
        wir die
        bekannte Version, um Rauschen zu vermeiden. (Resilienz-
        Fallback)
        """
        print("⚠️ FALLBACK AKTIVIERT: Blacklist wird statisch geladen
        (DB/API nicht erreichbar).")
        return {
            "kenntnisse", "fähigkeiten", "kommunikation", "deutsch",
            "englisch",
            "r", "bau", "ski", "sport", "medien", "wissenschaft",
            "erfahrung",
            "agil", "strategie", "prozess", "management", "analyse",
            "projektleitung",
            "kunden", "lösung", "team", "technik", "bereich",
            "verantwortung übernehmen",
            "beratung", "dienstleistungen", "informatik", "digitalisierung",
            "prägen", "datenschutz", "ethik", "gesundheit",
            "kommunizieren", "agiles"
        }

    # NEU: Resilienz-Fallback für Branchen-Mappings
    def _get_static_fallback_industry_mappings(self) -> Dict[str, str]:
        print("⚠️ FALLBACK AKTIVIERT: Branchen-Mappings werden
        statisch geladen.")
        # Diese statischen Mappings ersetzen die Logik aus
        organization_extractor.py
        return {
            'Informationstechnologie & Software':
            'IT|Software|Entwicklung|DevOps|Cloud|Informatik|Digitalisierung',
            'Finanzen & Versicherungen':
            'Finanz|Bank|Versicherung|Aktie|Treasury|Bilanz',
            'Automobil & Maschinenbau':
            'Automobil|Maschinenbau|Fertigung|Produktion|Anlage|Konstruktion',
            'Gesundheit & Soziales':
            'Klinik|Pflege|Sozial|Heim|Therapie|Krankenhaus',
            'Handel & Logistik': 'Handel|Logistik|Einzelhandel|Lager|Supply
            Chain',
            'Unbekannt/Generell':
            'Marketing|Vertrieb|Verwaltung|Assistenz'
        }

    # NEU: Resilienz-Fallback für Rollen-Mappings
    def _get_static_fallback_role_mappings(self) -> Dict[str, str]:
        print("⚠️ FALLBACK AKTIVIERT: Rollen-Mappings werden
        statisch geladen.")
        return {
            'Software-Entwicklung':
            'Entwickler|Developer|Programmierer|Coding|Frontend|Backend',
            'Data & Analytics': 'Data Scientist|Analyst|BI|Statistik',
            'Projektmanagement': 'Projektleiter|Scrum Master|Product
            Owner',
            'Führungskraft/Management': 'Leiter|Manager|Head of',
            'Unbekannt/Generell': 'Assistent|Kauffrau|Admin'
        }

```

Da ist schon **viel besser** als vorher  – aber du hast ein paar **fiese Logik-/Struktur-Probleme**, die dir genau diese „komische“ Fehlerkette + Reload-Crashes machen können.

Hier sind die wichtigsten Punkte (mit konkreten Fixes):

1) Du hast Methoden doppelt definiert (und überschreibst sie unbemerkt)

Du hast diese Methoden **jeweils zweimal** im selben File:

- `_get_static_fallback_industry_mappings`
- `_get_static_fallback_role_mappings`

In Python gilt: **die letzte Definition gewinnt**.

Das heißt: die ersten Mappings sind effektiv **tot** (und das kann auch IDE-Warnungen erklären).

 Fix: eine Version behalten, die andere löschen.

2) `_load_fallback_esco()` ruft `_generate_esco_from_csv(cache_file)` falsch auf

Du hast:

```
py

return self._generate_esco_from_csv(cache_file)
```

Aber `_generate_esco_from_csv(self)` nimmt **kein Argument**.

Das ist ein garantierter Crash, sobald `_load_fallback_esco()` benutzt wird.

 Code kopieren

 Fix:

```
py

return self._generate_esco_from_csv()
```

 Code kopieren

3) `get_esco_full()` iteriert blind über `raw_data`

Wenn Kotlin aus irgendeinem Grund **ein dict statt einer list** liefert, dann macht:

```
py

for item in raw_data:
```

bei dict → Iteration über Keys (strings) → dann knallt:

 Code kopieren

```
py

item.get(...)
```

weil `item` dann ein `str` ist.

 Code kopieren

 Fix: vor der Schleife normalisieren:

- wenn dict und enthält z.B. `"data"` oder `"items"` → daraus list machen
- sonst: wenn nicht list → fallback

4) Dein Fallback-Generator und `_load_or_create_fallback()` passen OK zusammen – aber dein `_load_fallback_esco()` ist inkonsistent

Du benutzt in `get_esco_full()`:

```
py

return self._load_or_create_fallback(filename, generation_method=self._generate_esco_from
```

Das ist gut.

 Code kopieren

Aber `_load_fallback_esco()` hat eine andere Logik und ist aktuell „kaputt“ (siehe Punkt 2).
Wenn du `_load_or_create_fallback()` eh sauber nutzt, kannst du `_load_fallback_esco()` entweder fixen oder entfernen.

5) Mini-Details, die später nerven

- `except (requests.exceptions.RequestException, ConnectionError) → ConnectionError` ist das Builtin. Besser nur `RequestException` oder `requests.exceptions.ConnectionError`.
- `from tqdm import tqdm` importierst du oben **und nochmal** innen. Nicht schlimm, aber redundant.
- Viele Imports (Set, List, Dict) nutzt du teils nicht → IDE meckert.

Gepatchte Version (kompakt, stabil, ohne doppelte Methoden)

Hier ist ein “drop-in” Patch, der die echten Crash-Kanten wegmacht:

```
python

import json
import os
from typing import Dict, Set, Any, List

import pandas as pd
import requests
from tqdm import tqdm

try:
    from app.core.constants import DEFAULT_KOTLIN_URL
except ImportError:
    DEFAULT_KOTLIN_URL = os.environ.get("KOTLIN_API_URL", "http://localhost:8080")

class KotlinRuleClient:
    def __init__(self, base_url: str | None = None):
        self.base_url = (base_url or DEFAULT_KOTLIN_URL).rstrip("/")
        self.fallback_path = "data/fallback_rules"
        self.esco_source = "data/esco"
        os.makedirs(self.fallback_path, exist_ok=True)
        print(f"🔗 KotlinRuleClient initialisiert. Basis-URL: {self.base_url}")

    def get_esco_full(self) -> List[Dict[str, Any]]:
        endpoint = f"{self.base_url}/api/v1/rules/esco-full"
        filename = "esco_fallback.json"

        try:
            print(f"🔄 Lade ESCO-Daten von: {endpoint}")
            response = requests.get(endpoint, timeout=15)
            response.raise_for_status()
            raw_data = response.json()

            # Robust: Kotlin kann list ODER dict liefern
```

```

if isinstance(raw_data, dict):
    # typische Patterns
    if "items" in raw_data and isinstance(raw_data["items"], list):
        raw_data = raw_data["items"]
    elif "data" in raw_data and isinstance(raw_data["data"], list):
        raw_data = raw_data["data"]
    else:
        print(f"⚠ ESCO: dict erhalten, kann nicht sinnvoll normalisieren. Key: {raw_data}")
        raise ValueError("ESCO response is dict, expected list.")

if not isinstance(raw_data, list):
    print(f"⚠ ESCO: Unerwarteter Typ: {type(raw_data)}")
    raise ValueError("ESCO response is not a list.")

normalized: List[Dict[str, Any]] = []

for item in tqdm(raw_data, desc="🚀 Lade Skills (Client)", unit=" sk", ncols=80):
    if not isinstance(item, dict):
        continue

    label = item.get("preferredLabel") or item.get("label") or item.get("esco_label")
    if not label:
        continue

    normalized.append({
        "label": label,
        "uri": item.get("esco_uri") or item.get("conceptUri") or item.get("uri"),
        "is_digital": bool(item.get("is_digital", False)),
        "level": item.get("level", 2),
    })

print(f"✅ {len(normalized)} Skills geladen & normalisiert.")
self._save_to_json(filename, normalized)
return normalized

except (requests.exceptions.RequestException, ValueError) as e:
    print(f"❌ ESCO API-Fehler ({e}). Nutze Fallback...")
    return self._load_or_create_fallback(filename, generation_method=self._generation_method)

def fetch_blacklist(self) -> Set[str]:
    endpoint = f"{self.base_url}/api/v1/rules/blacklist"
    filename = "blacklist_fallback.json"

    try:
        response = requests.get(endpoint, timeout=5)
        response.raise_for_status()
        data = response.json()
        self._save_to_json(filename, data)
        return set(data)
    except Exception as e:
        print(f"❌ API-Fehler bei Blacklist: {e}")
        data = self._load_or_create_fallback(filename, generation_method=self._generation_method)
        return set(data) if data else set()

def fetch_role_mappings(self) -> Dict[str, str]:
    endpoint = f"{self.base_url}/api/v1/rules/role-mappings"
    filename = "role_mappings_fallback.json"

    try:
        response = requests.get(endpoint, timeout=5)
        response.raise_for_status()
        data = response.json()
        self._save_to_json(filename, data)

```

```

        return data
    except Exception as e:
        print(f"❌ API-Fehler bei Role-Mappings: {e}")
        return self._load_or_create_fallback(filename, generation_method=self._get_static_fallback_method())

def fetch_industry_mappings(self) -> Dict[str, str]:
    endpoint = f"{self.base_url}/api/v1/rules/industry-mappings"
    filename = "industry_mappings_fallback.json"

    try:
        response = requests.get(endpoint, timeout=5)
        response.raise_for_status()
        data = response.json()
        self._save_to_json(filename, data)
        return data
    except Exception as e:
        print(f"❌ API-Fehler bei Industry-Mappings: {e}")
        return self._load_or_create_fallback(filename, generation_method=self._get_static_fallback_method())

def _load_or_create_fallback(self, filename: str, generation_method):
    path = os.path.join(self.fallback_path, filename)
    if not os.path.exists(path):
        print(f"⚠️ Fallback-Datei {filename} fehlt. Erstelle sie neu...")
        data = generation_method()
        self._save_to_json(filename, data)

    if os.path.exists(path):
        try:
            with open(path, "r", encoding="utf-8") as f:
                return json.load(f)
        except Exception as e:
            print(f"❌ Fehler beim Lesen von {path}: {e}")

    return [] if filename.endswith(".json") else {}

def _save_to_json(self, filename: str, data):
    path = os.path.join(self.fallback_path, filename)
    try:
        with open(path, "w", encoding="utf-8") as f:
            json.dump(data, f, ensure_ascii=False, indent=2)
    except Exception as e:
        print(f"⚠️ Konnte Fallback {filename} nicht speichern: {e}")

def _generate_esco_from_csv(self) -> List[Dict[str, Any]]:
    fallback_list: List[Dict[str, Any]] = []
    csv_file = os.path.join(self.esco_source, "skills.csv")

    if os.path.exists(csv_file):
        try:
            df = pd.read_csv(csv_file, usecols=["preferredLabel", "conceptUri"])
            for _, row in tqdm(df.iterrows(), desc="🔍 Indiziere Skills", unit=" sk",
                               leave=False):
                fallback_list.append({
                    "label": str(row["preferredLabel"]),
                    "uri": row["conceptUri"],
                    "is_digital": False,
                    "level": 2,
                })
        except Exception as e:
            print(f"⚠️ CSV-Fallback fehlgeschlagen: {e}")

    return fallback_list

def _get_static_fallback_blacklist_as_list(self) -> List[str]:

```



```
return list({
    "kenntnisse", "fähigkeiten", "kommunikation", "deutsch", "englisch",
    "team", "management", "analyse", "projektleitung", "erfahrung",
    "und", "oder", "bei", "mit", "job", "suche"
})

def _get_static_fallback_industry_mappings(self) -> Dict[str, str]:
    print("⚠ FALLBACK AKTIVIERT: Branchen-Mappings werden statisch geladen.")
    return {
        "Informationstechnologie & Software": "IT|Software|Entwicklung|DevOps|Cloud|In",
        "Finanzen & Versicherungen": "Finanz|Bank|Versicherung|Aktie|Treasury|Bilanz",
        "Automobil & Maschinenbau": "Automobil|Maschinenbau|Fertigung|Produktion|Anla",
        "Gesundheit & Soziales": "Klinik|Pflege|Sozial|Heim|Therapie|Krankenhaus",
        "Handel & Logistik": "Handel|Logistik|Einzelhandel|Lager|Supply Chain",
        "Unbekannt/Generell": "Marketing|Vertrieb|Verwaltung|Assistenz",
    }

def _get_static_fallback_role_mappings(self) -> Dict[str, str]:
    print("⚠ FALLBACK AKTIVIERT: Rollen-Mappings werden statisch geladen.")
    return {
        "Software-Entwicklung": "Entwickler|Developer|Programmierer|Coding|Frontend|Ba",
        "Data & Analytics": "Data Scientist|Analyst|BI|Statistik",
        "Projektmanagement": "Projektleiter|Scrum Master|Product Owner",
        "Führungskraft/Management": "Leiter|Manager|Head of",
        "Unbekannt/Generell": "Assistent|Kauffrau|Admin",
    }
```

Was du JETZT sofort testen kannst

1. Python startet ohne crash? (wichtig: einmal ohne reload)
2. `client.get_esco_full()` :
 - wenn Kotlin läuft: sollte normalisieren + cache schreiben
 - wenn Kotlin down: sollte CSV/json fallback ziehen ohne zu sterben

Wenn du mir kurz sagst, **welches Format Kotlin wirklich liefert** bei `/api/v1/rules/esco-full` (list? dict mit `data` ?), dann passe ich dir den Normalizer exakt darauf an, damit da nix mehr „geraten“ wird.



 **job_mining_workflow_...**
Python

 **job_mining_pipeline_v2...**
Python

 **jobmining_hybrid_fullst...**
Python

 **jobmining_hybrid_setu...**
Python

 **job_mining_workflow_...**
Python

 **jobminin workflow to op...**
Datei

 **Job Mining Architektur....**
Dokument

 **JobMiningWorkflowMan...**
Dokument

 **TXT.rtf**
Datei

aufgabe ich hae ähnlcihe code , der muss zu einer klasse usammen, ohne etwas vergessen.

Nachgedacht für 2m 42s >

Du hast bei dir **zwei "Workflow-Manager"-Welten**, die sich gerade gegenseitig in die Quere kommen:

- 1. **API/DI-Orchestrator** (TextExtractor → MetadataExtractor → Kotlin-Rule-Services → CompetenceExtractor → AnalysisResultDTO)
- 2. **File-System Workflow** (incoming/processing/archive/results/reports + JobMiningPipeline v2)

Wenn dann irgendwo die "falsche" Klasse importiert wird, bekommst du genau so Sachen wie `__init__() got unexpected keyword argument 'metadata_extractor'` oder fehlende Methoden wie `run_full_analysis_from_text`.

Hier ist **eine konsolidierte Klasse**, die **beides** abdeckt (ohne Feature-Verlust) und gleichzeitig **abwärtskompatible Alias-Methoden** bietet.

✔ Du kannst die Datei z.B. als `app/application/job_mining_workflow_manager.py` ablegen und alte Imports darauf umbiegen.

```
python

# app/application/job_mining_workflow_manager.py

from __future__ import annotations

import hashlib
import json
import logging
import shutil
from dataclasses import asdict, is_dataclass
from datetime import datetime
from pathlib import Path
from typing import Any, BinaryIO, Dict, List, Optional, Tuple, Union

logger = logging.getLogger(__name__)

# --- Optionale Imports (damit das File nicht sofort crasht, wenn du es isoliert testest)
try:
    from app.domain.models import AnalysisResultDTO, CompetenceDTO # Pydantic-DTOs (Python)
except Exception: # pragma: no cover
    AnalysisResultDTO = None # type: ignore
    CompetenceDTO = None # type: ignore

try:
    from app.core.constants import GLOBAL_BLACKLIST
except Exception: # pragma: no cover
    GLOBAL_BLACKLIST = set()

# Pipeline v2 (File-System Workflow)
try:
    from job_mining_pipeline_v2 import JobMiningPipeline, JobAnalysisResult
except Exception: # pragma: no cover
    JobMiningPipeline = None # type: ignore
    JobAnalysisResult = None # type: ignore

class JobMiningWorkflowManager:
    """
    Konsolidierter "Chef"-Manager:
    - API/DI-Orchestrator (Ebenen 1-7) → AnalysisResultDTO
    - File-System Workflow (incoming/processing/archive/results/reports) → JobMiningPipeline
    """
```

Motivation:

Du hattest mehrere ähnliche Klassenvarianten, die unterschiedliche `__init__` Signaturen und Methodennamen hatten (z.B. `metadata_extractor` vs. `meta_extractor`, `run_full_analysis`). Diese Klasse fasst alles zusammen und bietet Kompatibilitäts-Aliases.

```

"""

# -----
# 1) KONSTRUKTOR: DI-Orchestrator + optionaler File-System Workflow
# -----

def __init__(
    self,
    text_extractor: Optional[Any] = None,
    competence_extractor: Optional[Any] = None,
    organization_service: Optional[Any] = None,
    role_service: Optional[Any] = None,
    metadata_extractor: Optional[Any] = None,
    *,
    # File-System Workflow optional:
    base_dir: Optional[Union[str, Path]] = None,
    pipeline: Optional[Any] = None,
    enable_fs_workflow: bool = False,
):
    # --- Orchestrator-Dependencies (können bei purem FS-Workflow None sein) ---
    self.text_extractor = text_extractor
    self.competence_extractor = competence_extractor
    self.organization_service = organization_service
    self.role_service = role_service
    self.metadata_extractor = metadata_extractor # wichtig: einheitlicher Name

    # Backwards-Compat Aliases (falls alter Code noch self.meta_extractor nutzt)
    self.meta_extractor = metadata_extractor

    # --- FS Workflow setup ---
    self._fs_enabled = bool(enable_fs_workflow or base_dir or pipeline)
    self.base_dir: Optional[Path] = Path(base_dir) if base_dir else None

    if self._fs_enabled:
        if self.base_dir is None:
            # default Base Dir, falls du es aktivierst aber nichts übergibst
            self.base_dir = Path("job_mining_workflow")
        self.setup_directories()

        if pipeline is not None:
            self.pipeline = pipeline
        else:
            if JobMiningPipeline is None:
                raise RuntimeError(
                    "FS-Workflow aktiviert, aber JobMiningPipeline konnte nicht importiert werden."
                    "Stelle sicher, dass job_mining_pipeline_v2.py verfügbar ist."
                )
            self.pipeline = JobMiningPipeline()
        else:
            self.pipeline = pipeline # kann trotzdem gesetzt sein

# -----
# 2) ORCHESTRATOR ENTRYPOINTS (API / Batch / Scraper)
# -----

def run_full_analysis(self, file_stream: BinaryIO, filename: str) -> Any:
    """
    Standard-Workflow für Datei-Uploads (PDF/DOCX) über API.
    Erwartet: self.text_extractor + self.metadata_extractor + services + competence_extractor
    """

    self._require_orchestrator()

```

```

raw_text = self._extract_text(file_stream, filename)
cleaned_text = (raw_text or "").replace("\x00", "") # DB-safe

if not cleaned_text.strip():
    logger.error("Konnte keinen Text extrahieren: %s", filename)
    cleaned_text = "Extraktion fehlgeschlagen"

return self._run_analysis_from_text(cleaned_text, source_name=filename)

# Alias (falls du irgendwo async benutzt hast)
async def run_full_analysis_async(self, file_object: Any, filename: str) -> Any:
    """
    Async-Wrapper: akzeptiert auch UploadFile-like Objekte.
    """
    # UploadFile.file oder direkt file-like
    stream = getattr(file_object, "file", None) or file_object
    return self.run_full_analysis(stream, filename)

# Kompatibilitäts-Alias (wird in manchen Tests/Jobs genutzt)
def run_full_analysis_from_text(self, cleaned_text: str, source_name: str) -> Any:
    return self._run_analysis_from_text(cleaned_text.replace("\x00", ""), source_name)

def run_analysis_from_scraped_text(self, cleaned_text: str, source_name: str) -> Any:
    return self._run_analysis_from_text(cleaned_text.replace("\x00", ""), source_name)

# -----
# 3) ORCHESTRATOR CORE
# -----

def create_competence_dto(self, **kwargs) -> Optional[Any]:
    """
    Zentrale Factory (SSoT):
    - Blacklist-Filter
    - Level/Types absichern
    - Rückgabe als CompetenceDTO (wenn vorhanden), sonst Dict
    """
    term = (kwargs.get("original_term") or kwargs.get("label") or kwargs.get("name"))
    if term.lower() in GLOBAL_BLACKLIST:
        return None

    # Level absichern/normalisieren
    if "level" in kwargs:
        try:
            kwargs["level"] = int(kwargs["level"])
        except Exception:
            kwargs["level"] = 2

    # DTO bevorzugen, sonst dict zurückgeben (Pydantic kann dict meist trotzdem überne
    if CompetenceDTO is not None:
        try:
            return CompetenceDTO(**kwargs)
        except Exception as e:
            logger.error("DTO-Erstellung fehlgeschlagen für '%s': %s", term, e)
            return None

    return kwargs

def _determine_level(self, meta: Dict[str, Any]) -> int:
    """
    Business-Regel: finales wissenschaftliches Level bestimmen.
    (Du kannst das später schärfer machen – hier ist ein sicherer Default.)
    """
    inferred = meta.get("inferred_level")

```

```

    if inferred is not None:
        try:
            return int(inferred)
        except Exception:
            pass
    return 2

def _run_analysis_from_text(self, text: str, source_name: str) -> Any:
    self._require_orchestrator()

    # Ebene 7: Idempotenz
    raw_text_hash = hashlib.sha256(text.encode("utf-8")).hexdigest()

    # Ebene 6: Metadaten + Segmentierung (falls vorhanden)
    meta = self._extract_metadata(text, source_name)

    # Branche/Rolle über SSoT-Services
    industry = self._classify_industry(text)
    job_role = self._classify_role(text, meta.get("job_title", "") or meta.get("title", ""))

    # Segmentierter Text (z.B. nur Anforderungen) bevorzugen
    search_base = meta.get("processing_text") if meta.get("is_segmented") else text

    # Skills extrahieren (mit Kontext Rolle wenn möglich)
    raw_competences = self._extract_competences(search_base, job_role)

    # Optional: Level aus Kontext (Fachbuch/Uni/Discovery) setzen
    inferred_level = self._determine_level(meta)
    source_domain = meta.get("source_domain")

    normalized_competences = self._normalize_competences(
        raw_competences,
        inferred_level=inferred_level,
        source_domain=source_domain,
    )

    # DTO bauen (muss zu Kotlin/Jackson passen)
    payload = dict(
        title=meta.get("job_title") or meta.get("title") or source_name,
        job_role=job_role,
        region=meta.get("location") or meta.get("region") or "Unbekannt",
        industry=industry,
        posting_date=meta.get("posting_date") or meta.get("date") or "2024-01-01",
        raw_text_hash=raw_text_hash,
        raw_text=text,
        is_segmented=bool(meta.get("is_segmented", False)),
        competences=normalized_competences,
    )

    if AnalysisResultDTO is not None:
        try:
            return AnalysisResultDTO(**payload)
        except Exception as e:
            logger.error("AnalysisResultDTO Validierung fehlgeschlagen: %s", e)
            # Fallback: dict (besser als kompletter Crash)
            return payload

    return payload

# -----
# 4) ORCHESTRATOR HELPERS (robust gegen unterschiedliche Implementierungen)
# -----

def _require_orchestrator(self) -> None:

```

```

missing = []
if self.text_extractor is None:
    missing.append("text_extractor")
if self.competence_extractor is None:
    missing.append("competence_extractor")
if self.organization_service is None:
    missing.append("organization_service")
if self.role_service is None:
    missing.append("role_service")
if self.metadata_extractor is None:
    missing.append("metadata_extractor")

if missing:
    raise RuntimeError(
        "Orchestrator-Mode benötigt DI-Dependencies, aber fehlen: "
        + ", ".join(missing)
    )

def _extract_text(self, file_stream: BinaryIO, filename: str) -> str:
    # Unterstützt unterschiedliche Extractor-Interfaces
    if hasattr(self.text_extractor, "extract_text"):
        return self.text_extractor.extract_text(file_stream, filename)
    if hasattr(self.text_extractor, "extract_text_from_file"):
        return self.text_extractor.extract_text_from_file(file_stream, filename)
    raise AttributeError("text_extractor hat weder extract_text noch extract_text_from_file")

def _extract_metadata(self, text: str, source_name: str) -> Dict[str, Any]:
    ext = self.metadata_extractor
    if ext is None:
        return {}
    if hasattr(ext, "extract_all"):
        return ext.extract_all(text, source_name)
    if hasattr(ext, "extract_metadata"):
        return ext.extract_metadata(text)
    if hasattr(ext, "extract"):
        return ext.extract(text, source_name)
    return {}

def _classify_industry(self, text: str) -> str:
    svc = self.organization_service
    if svc is None:
        return "Unbekannt"
    if hasattr(svc, "classify_industry"):
        return svc.classify_industry(text)
    if hasattr(svc, "detect_industry"):
        return svc.detect_industry(text)
    return "Unbekannt"

def _classify_role(self, text: str, job_title: str) -> str:
    svc = self.role_service
    if svc is None:
        return "Unbekannt"
    if hasattr(svc, "classify_role"):
        return svc.classify_role(text, job_title)
    if hasattr(svc, "detect_role"):
        return svc.detect_role(job_title)
    return "Unbekannt"

def _extract_competences(self, text: str, job_role: str) -> Any:
    ext = self.competence_extractor
    if ext is None:
        return []
    # viele deiner Implementierungen erwarten (text, role) – andere nur (text)

```



```

try:
    return ext.extract_competences(text, job_role)
except TypeError:
    return ext.extract_competences(text)

def _normalize_competences(
    self,
    raw_competences: Any,
    *,
    inferred_level: int = 2,
    source_domain: Optional[str] = None,
) -> List[Any]:
    """
    Nimmt alles an (DTOs, dicts, dataclasses) und liefert eine saubere Liste:
    - Blacklist-Filter
    - Level/Source-Domain setzen (falls höherer Kontext)
    """

    if raw_competences is None:
        return []

    # Falls ein einzelnes Objekt kommt, in Liste packen
    if not isinstance(raw_competences, list):
        raw_list = [raw_competences]
    else:
        raw_list = raw_competences

    out: List[Any] = []
    for item in raw_list:
        if item is None:
            continue

        # dataclass -> dict
        if isinstance(item, dataclasses.DataClass):
            item = asdict(item)

        # Pydantic/BaseModel -> dict
        if isinstance(item, BaseModel):
            item = item.model_dump()

        if isinstance(item, dict):
            # Einheitliche Keys
            label = item.get("label") or item.get("name") or item.get("original_term")
            if str(label).strip().lower() in GLOBAL_BLACKLIST:
                continue

            # Context-Overwrites (Level 4/5 etc.)
            if inferred_level and int(inferred_level) > int(item.get("level", 2)) or 2:
                item["level"] = int(inferred_level)
            if source_domain:
                item["source_domain"] = source_domain

            dto = self.create_competence_dto(**item)
            if dto is not None:
                out.append(dto)
            continue

        # "Objekt"-Fall: versuche attribute zu setzen, wenn möglich
        try:
            label = getattr(item, "label", None) or getattr(item, "name", None) or getattr(item, "original_term", None)
            if label and str(label).lower() in GLOBAL_BLACKLIST:
                continue

            if inferred_level and getattr(item, "level", None):

```

```

        try:
            if int(inferred_level) > int(getattr(item, "level", 2) or 2):
                setattr(item, "level", int(inferred_level))
            if source_domain and hasattr(item, "source_domain"):
                setattr(item, "source_domain", source_domain)
        except Exception:
            pass

        out.append(item)
    except Exception:
        # im Notfall ignorieren
        continue

    return out

# -----
# 5) FILE-SYSTEM WORKFLOW (Pipeline v2) – 1:1 aus deinem v2.0 Manager
# -----

def setup_directories(self) -> None:
    self._require_fs()
    assert self.base_dir is not None

    self.dirs = {
        "incoming": self.base_dir / "incoming",
        "processing": self.base_dir / "processing",
        "archive": self.base_dir / "archive",
        "results": self.base_dir / "results",
        "reports": self.base_dir / "reports",
        "logs": self.base_dir / "logs",
    }
    for _, path in self.dirs.items():
        path.mkdir(parents=True, exist_ok=True)

def add_job_ad(self, file_path: str, metadata: Optional[Dict] = None) -> str:
    self._require_fs()
    source = Path(file_path)
    if not source.exists():
        raise FileNotFoundError(f"File not found: {file_path}")

    if source.suffix.lower() not in [".pdf", ".docx", ".doc"]:
        raise ValueError(f"Unsupported format: {source.suffix}")

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    new_filename = f"{timestamp}_{source.name}"
    destination = self.dirs["incoming"] / new_filename # type: ignore[attr-defined]

    shutil.copy2(source, destination)
    logger.info("Added job ad: %s", new_filename)

    if metadata:
        metadata_file = destination.with_suffix(".meta.json")
        with open(metadata_file, "w", encoding="utf-8") as f:
            json.dump(metadata, f, indent=2, ensure_ascii=False)

    return str(destination)

def process_incoming(self) -> List[Any]:
    self._require_fs()
    if self.pipeline is None:
        raise RuntimeError("FS-Workflow aktiv, aber pipeline ist None.")

    incoming_files = list(self.dirs["incoming"].glob("*")) # type: ignore[attr-defined]
    incoming_files = [f for f in incoming_files if f.suffix.lower() in [".pdf", ".docx", ".doc"]]

```

```

if not incoming_files:
    logger.info("No new job ads to process")
    return []

results: List[Any] = []

for file_path in incoming_files:
    processing_path = self.dirs["processing"] / file_path.name # type: ignore[attr-defined]
    try:
        # Move to processing
        shutil.move(str(file_path), str(processing_path))

        # Load metadata (if exists)
        metadata_file = file_path.with_suffix(".meta.json")
        custom_metadata: Dict[str, Any] = {}
        if metadata_file.exists():
            with open(metadata_file, "r", encoding="utf-8") as f:
                custom_metadata = json.load(f)
            shutil.move(str(metadata_file), str(self.dirs["processing"] / metadata_file.name))

        # Analyze
        result = self.pipeline.analyze_file(str(processing_path))

        # Attach metadata if the result supports it
        if custom_metadata:
            try:
                result.metadata = custom_metadata
            except Exception:
                pass

        results.append(result)

        # Move to archive
        month = datetime.now().strftime("%Y-%m")
        archive_subdir = self.dirs["archive"] / month # type: ignore[attr-defined]
        archive_subdir.mkdir(exist_ok=True)

        shutil.move(str(processing_path), str(archive_subdir / processing_path.name))

        # Move metadata too
        meta_processing = self.dirs["processing"] / metadata_file.name # type: ignore[attr-defined]
        if meta_processing.exists():
            shutil.move(str(meta_processing), str(archive_subdir / metadata_file.name))

    except Exception as e:
        logger.error("Error processing %s: %s", file_path.name, e)
        # Move failed file back to incoming
        if processing_path.exists():
            shutil.move(str(processing_path), str(self.dirs["incoming"] / processing_path.name))

return results

def save_results(self, results: List[Any]) -> None:
    self._require_fs()
    if self.pipeline is None:
        raise RuntimeError("FS-Workflow aktiv, aber pipeline ist None.")
    if not results:
        logger.warning("No results to save")
        return

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

```

```

json_path = self.dirs["results"] / f"results_{timestamp}.json" # type: ignore[attr-
csv_path = self.dirs["results"] / f"results_summary_{timestamp}.csv" # type: igno
csv_detailed_path = self.dirs["results"] / f"results_detailed_{timestamp}.csv" #

self.pipeline.export_to_json(results, str(json_path))
self.pipeline.export_to_csv(results, str(csv_path))
self.pipeline.export_detailed_csv(results, str(csv_detailed_path))

def generate_report(self, results: List[Any]) -> Optional[Tuple[Path, Path]]:
    self._require_fs()
    if self.pipeline is None:
        raise RuntimeError("FS-Workflow aktiv, aber pipeline ist None.")
    if not results:
        logger.warning("No results for report")
        return None

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

    text_report = self.pipeline.create_summary_report(results)
    text_path = self.dirs["reports"] / f"report_{timestamp}.txt" # type: ignore[attr-
    with open(text_path, "w", encoding="utf-8") as f:
        f.write(text_report)

    html_report = self._create_html_report(results, timestamp)
    html_path = self.dirs["reports"] / f"report_{timestamp}.html" # type: ignore[att-
    with open(html_path, "w", encoding="utf-8") as f:
        f.write(html_report)

    return text_path, html_path

def _create_html_report(self, results: List[Any], timestamp: str) -> str:
    # (1:1 kompatibel zu deiner v2.0 Logik – nutzt result.competence_count, result.ca
    total_jobs = len(results)
    total_skills = sum(getattr(r, "competence_count", 0) for r in results)
    avg_skills = (total_skills / total_jobs) if total_jobs else 0
    avg_conf = (sum(getattr(r, "avg_confidence", 0.0) for r in results) / total_jobs)

    all_categories: Dict[str, int] = {}
    for r in results:
        cats = getattr(r, "categories", {}) or {}
        for cat, count in cats.items():
            all_categories[cat] = all_categories.get(cat, 0) + int(count)

    top_categories = sorted(all_categories.items(), key=lambda x: x[1], reverse=True)

    all_skills: Dict[str, int] = {}
    for r in results:
        comps = getattr(r, "competences", []) or []
        for comp in comps:
            name = getattr(comp, "name", None) or (comp.get("name") if isinstance(comp
            if not name:
                continue
            all_skills[name] = all_skills.get(name, 0) + 1

    top_skills = sorted(all_skills.items(), key=lambda x: x[1], reverse=True)[:20]

    # Einfaches HTML (wie in deinem v2.0 Manager)
    html = f"""<!DOCTYPE html>

<html lang="en">
<head>
<meta charset="UTF-8">
<title>Job Mining Report {timestamp}</title>
<style>

```

```
body {{ font-family: Arial, sans-serif; margin: 24px; }}
h1 {{ margin-bottom: 4px; }}
.section {{ margin-top: 28px; }}
table {{ border-collapse: collapse; width: 100%; }}
th, td {{ border: 1px solid #ddd; padding: 8px; }}
th {{ background: #f5f5f5; text-align: left; }}
.small {{ color: #666; font-size: 0.9em; }}

</style>
</head>
<body>
<h1>Job Mining Report</h1>
<div class="small">Generated: {timestamp}</div>

<div class="section">
<h2><img alt="Flag icon" data-bbox="200 238 215 248"/> Summary</h2>
<ul>
<li>Total Jobs: {total_jobs}</li>
<li>Total Skills: {total_skills}</li>
<li>Avg Skills/Job: {avg_skills:.2f}</li>
<li>Avg Confidence: {avg_conf:.2%}</li>
</ul>
</div>

<div class="section">
<h2><img alt="Folder icon" data-bbox="200 388 215 398"/> Top Categories</h2>
<table>
<tr><th>Category</th><th>Count</th></tr>
{'.join(f"<tr><td>{c}</td><td>{n}</td></tr>" for c, n in top_categories)}
</table>
</div>

<div class="section">
<h2><img alt="Star icon" data-bbox="200 508 215 518"/> Top 20 Skills</h2>
<table>
<tr><th>Rank</th><th>Skill</th><th>Frequency</th><th>In % of Jobs</th></tr>
{'.join(f"<tr><td>{i}</td><td><b>{s}</b></td><td>{n}</td><td>{(n/total_jobs*100 if total_"}
</table>
</div>

<div class="section">
<h2><img alt="Document icon" data-bbox="200 628 215 638"/> Analyzed Job Ads</h2>
<ul>
{'.join(f"<li><b>{getattr(r,'filename','?')}</b> | Skills: {getattr(r,'competence_count'"}
</ul>
</div>

</body></html>
"""

    return html

def run_workflow(self) -> None:
    self._require_fs()
    results = self.process_incoming()
    if not results:
        logger.info("No new jobs to process. Workflow complete.")
        return
    self.save_results(results)
    self.generate_report(results)

def get_statistics(self) -> Dict[str, Any]:
    self._require_fs()
    archive_files = list(self.dirs["archive"].rglob("*")) # type: ignore[attr-defined]
    archive_files = [f for f in archive_files if f.suffix.lower() in [".pdf", ".docx"]]
```

```
result_files = list(self.dirs["results"].glob("results_*.json")) # type: ignore[untyped-call]
last_run = max([f.stat().st_mtime for f in result_files]) if result_files else None

return {
    "total_archived": len(archive_files),
    "total_results": len(result_files),
    "incoming_pending": len(list(self.dirs["incoming"].glob("*"))), # type: ignore[untyped-call]
    "last_run": last_run,
}

# -----
# 6) FS Guard
# -----

def _require_fs(self) -> None:
    if not getattr(self, "_fs_enabled", False):
        raise RuntimeError(
            "File-System Workflow ist nicht aktiviert. "
            "Initialisiere den Manager mit base_dir=... oder enable_fs_workflow=True."
        )
```

Was du damit sofort "reparierst"

- Ein einziges JobMiningWorkflowManager statt 2–3 Varianten, die sich in Signaturen unterscheiden
- metadata_extractor ist immer im init drin (das war bei dir der typische Crash)
- Alias-Methoden: run_full_analysis_from_text , run_analysis_from_scraped_text , optional run_full_analysis_async (für alte Aufrufer)
- FS-Workflow bleibt komplett (incoming/process/archive + save_results + report)

Wenn du willst, kann ich dir auch direkt sagen, welche deiner aktuellen Dateien du danach löschen/ersetzen solltest (damit nicht wieder "aus Versehen" die falsche Klasse importiert wird).



 KotlinRuleClient_check ...
Datei

 ProjectConfig.py
Python

 create_project.py
Python

 run_idempotency_chec...
Python

 env_check.py
Python

 check_paths.py
Python

 kotlin_rule_client.py
Python

 jobmining_hybrid_setu...
Python

 jobmining_hybrid_fullst...
Python

 evaluate_job_ads.py
Python

geht das besser ?

Nachgedacht für 26s >

Ja – **deutlich**. Im Moment hast du (mindestens) 3 Probleme, die dich später wieder beißen:

1. Mehrere KotlinRuleClient-Versionen mit unterschiedlichen Env-Variablen/Signaturen (z.B. KOTLIN_API_BASE_URL vs KOTLIN_API_URL) → führt zu „läuft lokal, crasht im anderen Kontext“.
2. Doppelte Methodennamen in deiner "ACTIVE"-Variante (_get_static_fallback_role_mappings und _get_static_fallback_industry_mappings sind doppelt) → Python nimmt nur die letzte Definition, die erste ist tot.  KotlinRuleClient_check versions

3. Du willst gleichzeitig **Resilienz** (nicht abstürzen) *und* **Testbarkeit** (Fehler sichtbar machen). Das geht sauber nur mit einem „strict mode“. Genau das wird auch in deinen Notizen schon vorgeschlagen.

So wird's „besser“ (SSoT + stabil + testbar)

Ziel

Eine Klasse `KotlinRuleClient`, mit:

- **einheitlicher** URL-Quelle (`DEFAULT_KOTLIN_URL` oder `KOTLIN_API_URL`)
- **einheitlicher** Return-Form (ESCO immer `List[Dict]` , Mappings immer `Dict[str,str]` , Blacklist immer `Set[str]`)
- **strict=False**: Runtime fällt auf Fallback zurück (kein Crash)
- **strict=True**: Tests brechen bewusst, damit du Probleme findest

KotlinRuleClient – saubere „Endversion“ (ohne Redundanz)

Du kannst das als Ersatz für deine jetzigen Mischversionen nehmen. Es deckt alles ab, was in deinen Versionen drin ist (ESCO, Blacklist, Role/Industry mappings, File-Fallback, CSV-Fallback). Die doppelten Methoden sind entfernt.
Passt zu der Idee aus deinen Docs („Resilienz + Testbarkeit“).

python

```
import json
import os
from pathlib import Path
from typing import Any, Dict, List, Set, Optional

import pandas as pd
import requests
from requests import Session

try:
    from app.core.constants import DEFAULT_KOTLIN_URL
except Exception:
    DEFAULT_KOTLIN_URL = os.environ.get("KOTLIN_API_URL", "http://localhost:8080")

class KotlinRuleClient:
    """
    Single Source of Truth für Rules/ESCO aus Kotlin.
    - strict=False (default): niemals Start crashen -> Fallback
    - strict=True: Fehler werden geworfen (ideal für Tests/CI)
    """

    def __init__(
        self,
        base_url: Optional[str] = None,
        *,
        strict: bool = False,
        timeout_s: int = 15,
        fallback_path: str = "data/fallback_rules",
        esco_source: str = "data/esco",
        session: Optional[Session] = None,
    ):
        self.base_url = (base_url or DEFAULT_KOTLIN_URL).rstrip("/")
        self.strict = strict
        self.timeout_s = timeout_s
```

```

self.fallback_dir = Path(fallback_path)
self.esco_source = Path(esco_source)

self.fallback_dir.mkdir(parents=True, exist_ok=True)
self._http = session or requests.Session()

# -----
# Public API
# -----

def get_esco_full(self) -> List[Dict[str, Any]]:
    endpoint = f"{self.base_url}/api/v1/rules/esco-full"
    cache_name = "esco_fallback.json"

    try:
        raw = self._get_json(endpoint, timeout=self.timeout_s)
        items = self._ensure_list(raw, endpoint)
        normalized = self._normalize_esco_items(items)
        self._save_json(cache_name, normalized)
        return normalized
    except Exception as e:
        return self._fallback_list(cache_name, e, generator=self._generate_esco_from_)

def fetch_blacklist(self) -> Set[str]:
    endpoint = f"{self.base_url}/api/v1/rules/blacklist"
    cache_name = "blacklist_fallback.json"

    try:
        data = self._get_json(endpoint, timeout=5)
        if not isinstance(data, list):
            raise ValueError(f"Blacklist ist nicht list, sondern {type(data)}")
        self._save_json(cache_name, data)
        return set(map(str, data))
    except Exception as e:
        fallback = self._fallback_list(cache_name, e, generator=self._static_blacklist_)
        return set(map(str, fallback))

def fetch_role_mappings(self) -> Dict[str, str]:
    endpoint = f"{self.base_url}/api/v1/rules/role-mappings"
    cache_name = "role_mappings_fallback.json"
    return self._fetch_mapping(endpoint, cache_name, self._static_role_mappings)

def fetch_industry_mappings(self) -> Dict[str, str]:
    endpoint = f"{self.base_url}/api/v1/rules/industry-mappings"
    cache_name = "industry_mappings_fallback.json"
    return self._fetch_mapping(endpoint, cache_name, self._static_industry_mappings)

# -----
# Internals
# -----

def _fetch_mapping(self, endpoint: str, cache_name: str, generator):
    try:
        data = self._get_json(endpoint, timeout=5)
        if not isinstance(data, dict):
            raise ValueError(f"Mapping ist nicht dict, sondern {type(data)}")
        self._save_json(cache_name, data)
        # normalisiere zu str:str
        return {str(k): str(v) for k, v in data.items()}
    except Exception as e:
        fallback = self._fallback_dict(cache_name, e, generator=generator)
        return {str(k): str(v) for k, v in (fallback or {}).items()}

```

```

def _get_json(self, url: str, timeout: int):
    resp = self._http.get(url, timeout=timeout)
    resp.raise_for_status()
    return resp.json()

def _ensure_list(self, raw: Any, endpoint: str) -> List[Any]:
    if isinstance(raw, list):
        return raw
    if isinstance(raw, dict):
        # häufige Patterns
        for key in ("items", "data", "results"):
            if key in raw and isinstance(raw[key], list):
                return raw[key]
        raise ValueError(f"ESCO Antwortformat unerwartet von {endpoint}: {type(raw)}")

def _normalize_esco_items(self, items: List[Any]) -> List[Dict[str, Any]]:
    out: List[Dict[str, Any]] = []
    for it in items:
        if not isinstance(it, dict):
            continue
        label = it.get("preferredLabel") or it.get("label") or it.get("esco_label")
        if not label:
            continue
        out.append(
            {
                "label": str(label),
                "uri": it.get("esco_uri") or it.get("conceptUri") or it.get("uri"),
                "is_digital": bool(it.get("is_digital", False)),
                "level": int(it.get("level", 2) or 2),
            }
        )
    return out

# -----
# Fallback handling (strict vs non-strict)
# -----

def _fallback_list(self, cache_name: str, err: Exception, generator):
    if self.strict:
        raise err
    path = self.fallback_dir / cache_name
    if not path.exists():
        data = generator()
        self._save_json(cache_name, data)
    return self._load_json_list(cache_name)

def _fallback_dict(self, cache_name: str, err: Exception, generator):
    if self.strict:
        raise err
    path = self.fallback_dir / cache_name
    if not path.exists():
        data = generator()
        self._save_json(cache_name, data)
    return self._load_json_dict(cache_name)

def _save_json(self, filename: str, data: Any):
    path = self.fallback_dir / filename
    with open(path, "w", encoding="utf-8") as f:
        json.dump(data, f, ensure_ascii=False, indent=2)

def _load_json_list(self, filename: str) -> List[Any]:
    path = self.fallback_dir / filename
    if not path.exists():

```

```
        return []
    with open(path, "r", encoding="utf-8") as f:
        data = json.load(f)
    return data if isinstance(data, list) else []

def _load_json_dict(self, filename: str) -> Dict[str, Any]:
    path = self.fallback_dir / filename
    if not path.exists():
        return {}
    with open(path, "r", encoding="utf-8") as f:
        data = json.load(f)
    return data if isinstance(data, dict) else {}

# -----
# Generators
# -----

def _generate_esco_from_csv(self) -> List[Dict[str, Any]]:
    """
    CSV-Fallback: data/esco/skills.csv mit preferredLabel + conceptUri
    """
    out: List[Dict[str, Any]] = []
    csv_file = self.esco_source / "skills.csv"
    if not csv_file.exists():
        return out

    df = pd.read_csv(csv_file, usecols=["preferredLabel", "conceptUri"])
    for _, row in df.iterrows():
        out.append(
            {"label": str(row["preferredLabel"]), "uri": row["conceptUri"], "is_digit":
            }
        )
    return out

def _static_blacklist(self) -> List[str]:
    return [
        "kenntnisse", "fähigkeiten", "kommunikation", "deutsch", "englisch",
        "team", "management", "analyse", "projektleitung", "erfahrung",
        "und", "oder", "bei", "mit", "job", "suche",
    ]

def _static_industry_mappings(self) -> Dict[str, str]:
    return {
        "Informationstechnologie & Software": "IT|Software|Entwicklung|DevOps|Cloud|In",
        "Finanzen & Versicherungen": "Finanz|Bank|Versicherung|Aktie|Treasury|Bilanz",
        "Automobil & Maschinenbau": "Automobil|Maschinenbau|Fertigung|Produktion|Anla",
        "Gesundheit & Soziales": "Klinik|Pflege|Sozial|Heim|Therapie|Krankenhaus",
        "Handel & Logistik": "Handel|Logistik|Einzelhandel|Lager|Supply Chain",
        "Unbekannt/Generell": "Marketing|Vertrieb|Verwaltung|Assistenz",
    }

def _static_role_mappings(self) -> Dict[str, str]:
    return {
        "Software-Entwicklung": "Entwickler|Developer|Programmierer|Coding|Frontend|B",
        "Data & Analytics": "Data Scientist|Analyst|BI|Statistik",
        "Projektmanagement": "Projektleiter|Scrum Master|Product Owner",
        "Führungskraft/Management": "Leiter|Manager|Head of",
        "Unbekannt/Generell": "Assistent|Kauffrau|Admin",
    }
```

Warum ist das konkret besser als deine aktuelle "ACTIVE"-Datei?

- Keine doppelten Methoden (bei dir überschreiben sich Funktionen unbemerkt).

KotlinRuleClient_check versions
- Einheitliche "Quelle der Wahrheit" für URL (bei dir mischen sich Env-Namen über mehrere Versionen).
- **strict mode:**
 - `strict=False` = Produktion/Dev: **läuft weiter** (Fallback)
 - `strict=True` = Tests: **knallt bewusst**, damit du Probleme findestDas ist genau die Trennung „Resilienz vs. Testbarkeit“, die du brauchst.

Extra: passt zu deinem Repo/Architektur-Plan

Deine `ProjectConfig` ist genau dafür gedacht: zentrale Pfade/Configs/Repository Pattern statt Hardcoding.

ProjectConfig

Du kannst z.B. `fallback_path` / `esco_source` direkt aus Config durchreichen und musst nix im Code suchen.

Und dein SpaCyExtractor?

Dein Ausschnitt ist okay als Start, aber „besser“ wäre:

- **kein globales** `NLP = spacy.load(...)` **beim Import** (das macht Tests & Uvicorn reload kaputt),
- Repo liefert **eine** Methode `get_all_identifiable_labels()` (du hast das in deinen Notizen schon als "Interface-Vertrag" stehen).
- Baue beim Init ein dict `label_lower -> (uri, id)` statt jedes Mal `next(... for c in get_all_competences())` (das ist sonst O(n) pro Treffer).

Wenn du willst, poste den Rest deiner `SpaCyCompetenceExtractor` -Klasse, dann baue ich sie dir genauso „endgültig“ um (lazy init + Repo-Vertrag + schneller lookup) – ohne dass du Features verlierst.

Quellen

     

 **1109c4e0-0e89-4305-...**
Datei

projectconfig gibtes die ? ich glaubwar aus alten programm . ich habe ein . aber was wäre besser ?

Nachgedacht für 21s >

Ja – **ProjectConfig** gib’t bei dir wirklich. Das ist nicht nur “aus dem alten Programm”, sondern eine komplette **Refactor-Config-Datei** als `@dataclass` mit Defaults, `validate()` und `load_config_from_file()` usw.

ABER: In deinem aktuellen Setup (FastAPI + Kotlin-Bridge) nutzt du stattdessen überall **hardcoded** `BASE_DATA_DIR` + `PATHS` in `main.py` /Notizen.

Darum wirkt ProjectConfig wie “alt”, weil es gerade **nicht sauber eingebunden** ist.

Was ist “besser”?

Kommt drauf an, was du willst:

Option A (okay): ProjectConfig behalten, aber auf “heute” trimmen

ProjectConfig ist grundsätzlich brauchbar, **wenn** du:

- nur die Felder behältst, die du wirklich nutzt (base_data_dir, domains/esco/jobs, kotlin_api_url, fallback dirs)
- `validate()` so anpasst, dass es **deine echten Ordner/Dateien** prüft (z.B. `data/job_domains/*.json`, `data/esco/skills.csv` usw.)
- NICHT mehr 3 verschiedene Stellen hast, die Pfade definieren

ProjectConfig in der aktuellen Form hat noch viele "Analyse/Dashboard/AI"-Felder, die bei deinem Live-Service gerade gar nicht gebraucht werden.

Option B (meist besser für FastAPI): Settings via ENV/.env (Pydantic Settings)

Für Services ist das meistens die sauberste Lösung:

- du kannst auf Mac/Docker/Server nur per Env umstellen
- du vermeidest hart kodierte Mac-Pfade wie `/Users/...`  `bug rule_industrie_api`
- du hast "Single Source of Truth" für `KOTLIN_API_URL`, `BASE_DATA_DIR`, fallback dirs usw.

Beispiel (kurz & robust):

```
python

# app/core/settings.py
from pathlib import Path
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8")

    base_data_dir: Path = Path("data")
    kotlin_api_url: str = "http://localhost:8080"

    @property
    def paths(self) -> dict:
        return {
            "ESCO_DATA": self.base_data_dir / "esco",
            "DOMAINS": self.base_data_dir / "job_domains",
            "JOBS": self.base_data_dir / "jobs",
            "BOOKS": self.base_data_dir / "source_pdfs" / "fachbuecher",
            "ACADEMIA": self.base_data_dir / "source_pdfs" / "modulhandbuecher",
        }

settings = Settings()
```

In `main.py` dann:

```
python

from app.core.settings import settings

PATHS = settings.paths
RULE_CLIENT = KotlinRuleClient(base_url=settings.kotlin_api_url)
```

Und in `.env` :

```
ini

BASE_DATA_DIR=/Users/xxx/.../python-backend/data
KOTLIN_API_URL=http://localhost:8080
```

So hast du **kein** "wo muss ich jetzt wieder den Pfad ändern?" mehr.

Und zu deiner Frage: “Wie kommt ESCO da rein?”

 Code kopieren

Bei dir gibt’s 2 gültige Wege (du solltest **einen** wählen):

Weg 1: ESCO lokal aus Datei

Repo bekommt `esco_master.json` oder `skills.csv` aus `data/esco/...`

So wie du’s in mehreren Snippets hast.

Weg 2 (SSoT, empfehlenswert): ESCO von Kotlin syncen + Domains lokal laden

Das ist genau der Ansatz, den du schon als “SSoT Sync” notiert hast:

- `sync_from_ssot()` holt Ebene 2/3 von Kotlin
- `load_local_domains()` lädt Ebene 4/5 aus JSON

Dann muss dein `HybridCompetenceRepository.__init__` genau **die Parameter** akzeptieren, die du in `main.py` übergibst – sonst kommt der Klassiker:

unexpected keyword argument 'fachbuch_path'  TXT

Mein Vorschlag (damit du nicht wieder 3 Versionen hast)

- **Nimm Option B (Settings via ENV/.env)** für den Service.
- Lass `ProjectConfig` *entweder* ganz weg *oder* nutze es nur noch für “Offline Batch / Auswertung / Experimente”.
- Mach **eine** klare Repo-Signatur (z.B. `kotlin_api_url`, `fachbuch_path`, `academia_path`) und nutze die überall gleich – dann ist der `TypeError` weg.

Wenn du mir sagst, welche Betriebsart du willst:

1. “Service muss immer laufen, auch ohne Kotlin” (Fallback-first)
oder
2. “Service darf failen, wenn Kotlin fehlt” (strict)

...dann gebe ich dir die **eine** finale Version von:

- `settings.py`
- `main.py` Bootstrap
- `HybridCompetenceRepository.__init__` + `sync/load`

so dass alles **zusammenpasst** und du nicht mehr “alt vs neu” jonglierst.

Quellen

     ...

1, für eigene tests also weg 2 oder ?

Für **eigene Tests** ist **Weg 2 (Settings via .env + optional strict mode)** am besten, ja.

Der Punkt ist: Du willst lokal schnell umstellen können (andere Pfade, anderer Kotlin-Port), ohne jedes Mal Code zu ändern. Das macht `.env` /ENV perfekt. Und für Tests willst du manchmal **strict** (damit Fehler knallen), manchmal **fallback** (damit der Workflow weiterläuft).

Empfehlung für dich

- **Config: Weg 2 (.env / Settings)**
- **KotlinRuleClient: default** `strict=False` (läuft immer weiter)
- Für “echte” Tests/CI: `strict=True` (damit du sofort siehst, wenn Kotlin/ESCO kaputt ist)

Minimal-Setup (so dass du heute loslegen kannst)

1) app/core/settings.py

```
python

from pathlib import Path
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8")

    base_data_dir: Path = Path("data")
    kotlin_api_url: str = "http://localhost:8080"

    # Test/Debug toggles
    strict_rules: bool = False

    @property
    def paths(self) -> dict:
        return {
            "ESCO_DATA": self.base_data_dir / "esco",
            "DOMAINS": self.base_data_dir / "job_domains",
            "JOBS": self.base_data_dir / "jobs",
            "FALLBACK_RULES": self.base_data_dir / "fallback_rules",
        }

settings = Settings()
```

2) .env (im Projekt-Root)

```
env

BASE_DATA_DIR=/Users/<du>/.../python-backend/data
KOTLIN_API_URL=http://localhost:8080
STRICT_RULES=false
```

3) Client so initialisieren (z.B. in main.py oder Repository)

```
python

from app.core.settings import settings
from app.infrastructure.clients.kotlin_rule_client import KotlinRuleClient

rule_client = KotlinRuleClient(
    base_url=settings.kotlin_api_url,
    strict=settings.strict_rules,
    fallback_path=str(settings.paths["FALLBACK_RULES"]),
    esco_source=str(settings.paths["ESCO_DATA"]),
)
```

Was ist mit ProjectConfig?

 Code kopieren

Für deinen Fall: würde ich weglassen (oder nur für alte Offline-Skripte behalten), weil du sonst wieder zwei Config-Systeme hast.

Daumenregel

 Code kopieren

- eigene lokale Tests, viel rumprobieren → .env + strict_rules toggle 
- Pipeline soll "immer laufen" → strict_rules=false 

- du willst Fehler sofort sehen → `strict_rules=true` 

Wenn du willst, kann ich dir auch kurz sagen, **welche deiner vorhandenen Dateien** du am besten "deprecate"/löschen solltest, damit nicht aus Versehen wieder die alte Config gezogen wird (ProjectConfig, PATHS in main, etc.).



das soll schon docker tauglich werdne. alles.

Nachgedacht für 38s >

Ja – **Docker-tauglich für "alles"** geht, aber nur wenn du konsequent **(a) keine Festplatten-Pfade** mehr im Code hast und **(b) alles über ENV + Volumes** verdrahtest.

Du hast aktuell in deinen Notizen/ `main.py` -Entwurf noch harte Mac-Pfade (`/Users/.../python-backend/data`). Das ist genau der Grund, warum's beim Docker/Cloud-Move kaputt geht.

Und du hast schon die Richtung "relative Pfade ab Repo" beschrieben – das ist richtig.

Unten ist ein Setup, das **für lokales Dev + Docker + später Cloud** sauber funktioniert.

1) Eine Regel: Daten immer unter `/data` im Container

Im Container willst du nicht "wo liegt das Repo?", sondern einfach:

- `BASE_DATA_DIR=/data`
- und du mountest dein `./python-backend/data` nach `/data`

Damit ist alles reproduzierbar.

`app/core/settings.py` (SSoT für Pfade + URLs)

```
python

from pathlib import Path
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8")

    # Basis für ALLE relativen Datenpfade
    base_data_dir: Path = Path("data")

    # Kotlin in Docker: http://kotlin-api:8080 (Service-Name!)
    kotlin_api_url: str = "http://localhost:8080"

    # Für "läuft immer weiter" vs "Tests sollen knallen"
    strict_rules: bool = False

    # Optional: DB für Python (wenn Python direkt DB nutzt)
    database_url: str | None = None

    @property
    def paths(self) -> dict:
        bd = self.base_data_dir
        return {
            "ESCO_DATA": bd / "esco",
            "DOMAINS": bd / "job_domains",
            "JOBS": bd / "jobs",
            "FALLBACK_RULES": bd / "fallback_rules",
```

```
        "SOURCE_PDFS_BOOKS": bd / "source_pdfs" / "fachbuecher",
        "SOURCE_PDFS_ACADEMIA": bd / "source_pdfs" / "modulhandbuecher",
    }

    settings = Settings()
```

In `main.py` / **Bootstrap: keine OS-Pfad-Bastelei mehr**

Du nutzt nur noch `settings.paths` :

```
python

from app.core.settings import settings
from app.infrastructure.clients.kotlin_rule_client import KotlinRuleClient

PATHS = settings.paths

rule_client = KotlinRuleClient(
    base_url=settings.kotlin_api_url,
    strict=settings.strict_rules,
    fallback_path=str(PATHS["FALLBACK_RULES"]),
    esco_source=str(PATHS["ESCO_DATA"]),
)
```

Damit sind deine 3 relevanten Ordner exakt das, was du beschrieben hast (`data/job_domains` , `data/jobs` , `data/esco`) – nur ohne Mac-Pfad.

2) Dockerfile für Python (inkl. Playwright – docker-tauglich)

Leg das in `python-backend/Dockerfile` (nicht im Repo-Root, außer du willst es bewusst):

```
dockerfile

FROM python:3.11-slim

# System libs für Playwright + ggf. Postgres client libs
RUN apt-get update && apt-get install -y \
    gcc curl \
    libpq-dev \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Playwright Browser + Linux deps
RUN playwright install chromium --with-deps

COPY . .

# Im Container soll er auf 0.0.0.0 hören
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Das ist im Prinzip das, was du in deinen Notizen schon als “richtiges Dockerfile” stehen hattest – nur sauber platziert.  `TEXT`

3) docker-compose.yml (alles: Postgres + Kotlin + Python)

Die Datei gehört ins **Repo-Root** (damit du `docker compose up` von oben starten kannst).

Wichtigster Docker-Punkt: **innerhalb von Docker ist localhost immer der Container selbst.**
Python erreicht Kotlin nicht via `localhost:8080`, sondern via **Service-Name** `kotlin-api:8080`.

yaml

```
services:
  jobmining-db:
    image: postgres:15-alpine
    container_name: jobmining-db
    environment:
      POSTGRES_DB: jobmining_db
      POSTGRES_USER: jobmining
      POSTGRES_PASSWORD: jobmining
    ports:
      - "5432:5432"
    volumes:
      - db_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U jobmining -d jobmining_db"]
      interval: 5s
      timeout: 5s
      retries: 20

  kotlin-api:
    # entweder ein Image oder build aus deinem Kotlin-Projekt
    build:
      context: ./kotlin-backend
    environment:
      DB_HOST: jobmining-db
      DB_PORT: 5432
      DB_NAME: jobmining_db
      DB_USER: jobmining
      DB_PASSWORD: jobmining
    ports:
      - "8080:8080"
    depends_on:
      jobmining-db:
        condition: service_healthy

  python-api:
    build:
      context: ./python-backend
    environment:
      BASE_DATA_DIR: /data
      KOTLIN_API_URL: http://kotlin-api:8080
      STRICT_RULES: "false"
      # optional, falls Python DB braucht:
      # DATABASE_URL: postgresql+psycopg2://jobmining:jobmining@jobmining-db:5432/jobmining_db
    ports:
      - "8000:8000"
    volumes:
      # deine Daten (ESCO, domains, jobs, fallback_rules) bleiben persistent & editierbar
      - ./python-backend/data:/data
    depends_on:
      kotlin-api:
        condition: service_started

volumes:
  db_data:
```

Was das für dich löst:

- Keine "Port 8080 refused"-Fehler, weil Python nicht mehr `localhost` nimmt (innerhalb Docker wäre das falsch).
- Kotlin startet erst, wenn DB wirklich ready ist (healthcheck).
- Daten liegen persistent in `./python-backend/data` und sind im Container unter `/data` verfügbar (genau das wollen wir).

4) "Nur DB in Docker" (Dev-komfortabel) – extra Compose

Du hattest das auch schon als bevorzugten Dev-Weg beschrieben (nur DB in Docker, Rest lokal).

Dann mach eine `docker-compose.dev.yml`:

```
yaml

services:
  jobmining-db:
    image: postgres:15-alpine
    container_name: jobmining-db
    environment:
      POSTGRES_DB: jobmining_db
      POSTGRES_USER: jobmining
      POSTGRES_PASSWORD: jobmining
    ports:
      - "5432:5432"
    volumes:
      - db_data:/var/lib/postgresql/data

volumes:
  db_data:
```

Start:

```
bash

docker compose -f docker-compose.dev.yml up -d
```

Dann lokal:

 Code kopieren

- Kotlin: verbindet auf `localhost:5432`
- Python: `KOTLIN_API_URL=http://localhost:8080` (wenn Kotlin lokal läuft)

5) "Was wäre besser als ProjectConfig?"

Du hast eine `ProjectConfig.py`, aber die ist stark auf ein altes/anderes "refactored analysis pipeline"

Setup mit `data/competences/...` ausgelegt. Das kollidiert mit deinem aktuellen "3 Dom  Code kopieren

Für Docker ist **Pydantic Settings + .env** fast immer besser, weil:

- 12-Factor Style (Cloud-Standard)
- kein "Config-Code" der beim Import schon Ordner baut (das macht Docker-Builds ur  Code kopieren
- du kannst alles per compose `environment:` setzen

Du kannst `ProjectConfig.py` trotzdem behalten – aber eher als **Offline-Batch config**, nicht als Runtime-Config für FastAPI.

6) Kommandos, die du wirklich brauchst

Alles starten (build + run):

```
bash
```



```
docker compose up --build -d
```

Logs schauen:

Code kopieren

```
bash

docker compose logs -f python-api
docker compose logs -f kotlin-api
docker compose logs -f jobmining-db
```

Stoppen + alles runter:

```
bash

docker compose down
```

DB resettten (Vorsicht: Daten weg):

Code kopieren

```
bash

docker compose down -v
```

Code kopieren

Wenn du willst, mache ich dir das “komplett rund” mit 2 gezielten Änderungen:

- 1. Ich passe deinen **KotlinRuleClient** so an, dass er in Docker automatisch `KOTLIN_API_URL=http://kotlin-api:8080` nimmt und lokal `localhost` – ohne dass du irgendwas umschaltest. (Du hast ja mehrere Versionen davon.)
- 2. Ich gebe dir eine **kleine Dockerfile -Vorlage für Kotlin** (Gradle multi-stage), damit du nicht in IntelliJ bauen musst.

Sag nur kurz: Läuft Kotlin bei dir als eigenes Projekt in `./kotlin-backend/` (so wie im compose oben angenommen), oder ist das ein anderes Verzeichnis?

Quellen

< 2/2 >      ...

ja

Perfekt — dann machen wir’s jetzt **sauber Docker-tauglich** (Kotlin + Python + DB), ohne dass du später wieder an 10 Stellen Pfade/URLs drehen musst.

1) Kotlin: Dockerfile (multi-stage, schnell, production-like)

Lege das hier in `./kotlin-backend/Dockerfile` :

```
dockerfile

# ----- Build stage -----
FROM gradle:8.7-jdk17 AS build
WORKDIR /home/gradle/project

# Nur Build-Dateien zuerst kopieren (bessere Docker-Cache-Nutzung)
COPY build.gradle* settings.gradle* gradle.properties gradlew ./
COPY gradle ./gradle

# Dependencies “warm” ziehen (Cache)
RUN ./gradlew --no-daemon dependencies || true
```

```
# Jetzt den Source-Code kopieren
COPY . .

# Build (Spring Boot fat jar)
RUN ./gradlew --no-daemon clean bootJar

# ----- Runtime stage -----
FROM eclipse-temurin:17-jre
WORKDIR /app

# Optional: non-root user
RUN useradd -m appuser
USER appuser

# Jar rein
COPY --from=build /home/gradle/project/build/libs/*.jar /app/app.jar

EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Kotlin: .dockerignore (wichtig!)

In `./kotlin-backend/.dockerignore`:

```
csharp

.gradle
build
out
.idea
*.iml
.DS_Store
```

2) Kotlin: application config für Docker (DB Host = Service-Name)

In Docker ist DB nicht `localhost`, sondern `jobmining-db`.

Wenn du Spring Boot nutzt, ist das Minimum:

 Code kopieren

`application.yml` (Beispiel)

```
yaml

server:
  port: 8080

spring:
  datasource:
    url: jdbc:postgresql://${DB_HOST:localhost}:${DB_PORT:5432}/${DB_NAME:jobmining_db}
    username: ${DB_USER:jobmining}
    password: ${DB_PASSWORD:jobmining}

management:
  endpoints:
    web:
      exposure:
        include: health,info
```

Wenn du kein actuator willst: ok, aber Healthcheck wird dann ein anderer Endpoint.

3) docker-compose.yml: final (Service-Name statt localhost)

Hier ein "richtiges" Compose, das du im Repo-Root benutzen kannst:

```
yaml

services:
  jobmining-db:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: jobmining_db
      POSTGRES_USER: jobmining
      POSTGRES_PASSWORD: jobmining
    ports:
      - "5432:5432"
    volumes:
      - db_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U jobmining -d jobmining_db"]
      interval: 5s
      timeout: 5s
      retries: 20

  kotlin-api:
    build:
      context: ./kotlin-backend
    environment:
      DB_HOST: jobmining-db
      DB_PORT: 5432
      DB_NAME: jobmining_db
      DB_USER: jobmining
      DB_PASSWORD: jobmining
    ports:
      - "8080:8080"
    depends_on:
      jobmining-db:
        condition: service_healthy
    # Healthcheck: bevorzugt actuator
    healthcheck:
      test: ["CMD-SHELL", "wget -qO- http://localhost:8080/actuator/health | grep -q UP"]
      interval: 10s
      timeout: 5s
      retries: 30

  python-api:
    build:
      context: ./python-backend
    environment:
      BASE_DATA_DIR: /data
      KOTLIN_API_URL: http://kotlin-api:8080
      STRICT_RULES: "false"
    ports:
      - "8000:8000"
    volumes:
      - ./python-backend/data:/data
    depends_on:
      kotlin-api:
        condition: service_healthy

volumes:
  db_data:
```

Wenn du KEIN actuator hast

Dann ersetze den Kotlin-Healthcheck z.B. durch einen deiner Rules-Endpunkte (den es sicher gibt), z.B.:

```
yaml

healthcheck:
  test: ["CMD-SHELL", "wget -q0- http://localhost:8080/api/v1/rules/blacklist >/dev/null"]
  interval: 10s
  timeout: 5s
  retries: 30
```

4) Python: die eine Regel für Docker-Fitness

Kein `"/Users/..."` im Code.

Code kopieren

Nur:

- `BASE_DATA_DIR=/data`
- Pfade immer relativ zu `base_data_dir`

Dann läuft es:

- lokal (wenn `BASE_DATA_DIR=data`)
- in Docker (wenn `BASE_DATA_DIR=/data`)

5) Quick Commands (so arbeitest du dann)

```
bash

docker compose up --build -d
docker compose logs -f kotlin-api
docker compose logs -f python-api
```

Test:

```
bash

curl http://localhost:8080/actuator/health
curl http://localhost:8000/docs
```

6) Ein kleiner “Trick”, damit du lokal UND Docker ohne Code-Änderung runst

Code kopieren

In `.env` lokal:

```
ini

KOTLIN_API_URL=http://localhost:8080
BASE_DATA_DIR=./python-backend/data
```

Code kopieren

In `docker-compose.yml` überschreibst du es dann für Container:

```
ini

KOTLIN_API_URL=http://kotlin-api:8080
BASE_DATA_DIR=/data
```

Code kopieren

Damit ist die App überall gleich – nur die Umgebung ist anders.

Wenn du willst, paste ich dir als nächstes auch noch eine **Kotlin Gradle Build Optimierung** (layer caching + build args) und eine **Healthcheck-Route ohne actuator** (kleiner Controller), damit du die actuator-

Abhängigkeit nicht brauchst.

